

3DoF Bolt Sorter: Computer Vision Pipeline

June 9, 2026

1 System Overview

The computer vision pipeline has two main jobs:

1. **Detect and measure each screw** — find where it is in the camera frame and determine its physical size (e.g., M6, M8, M10).
2. **Locate it in 3D space** — convert its pixel position into real-world coordinates so the robot arm can pick it up.

The sections below walk through each part of this process step by step.

2 Key Components

2.1 The ArUco Reference Marker

A camera sees the world in pixels, which have no inherent physical size. To convert pixel measurements into real-world millimeters, the system uses an **ArUco marker**, a small printed square with a known physical size ($s = 20.6$ mm) placed in the camera’s field of view.

The marker serves two purposes:

1. **Pixel-to-millimeter scaling:** By measuring how many pixels wide the marker appears, the system computes a scale factor κ :

$$\kappa = \frac{s_{\text{mm}}}{W_{\text{px}}} \quad [\text{mm/pixel}] \quad (1)$$

This factor is then applied to all other measurements in the frame.

2. **Spatial reference point:** The center of the marker acts as a known anchor position in the robot’s coordinate system, allowing the system to calculate where each screw is relative to the robot base.

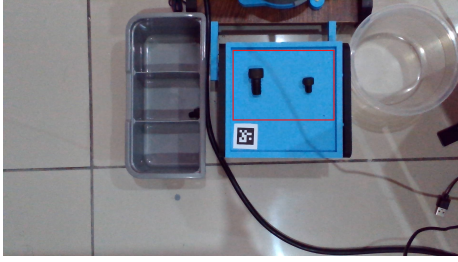
Because κ is computed fresh for every frame, the pipeline automatically adapts if the camera height changes slightly.

2.2 Detection Region of Interest

Rather than running the object detector on the entire 1280×720 frame, the system limits detection to a pre-defined **Region of Interest (ROI)** — a smaller rectangular window where screws are expected to appear. This reduces noise from irrelevant parts of the scene and improves detection reliability.

The YOLO model scans the ROI and returns a bounding box for each detected screw:

$$\text{BBox} = [u_{\min}, v_{\min}, u_{\max}, v_{\max}] \quad (2)$$



(a) Full camera frame



(b) Cropped ROI

Figure 1: The full camera frame (left) and the cropped ROI region passed to the detector (right).

3 Part 1: Screw Detection and Size Classification

This section describes the full processing chain that takes a raw camera frame and outputs a physical size label for each detected screw.

3.1 Processing Pipeline Overview

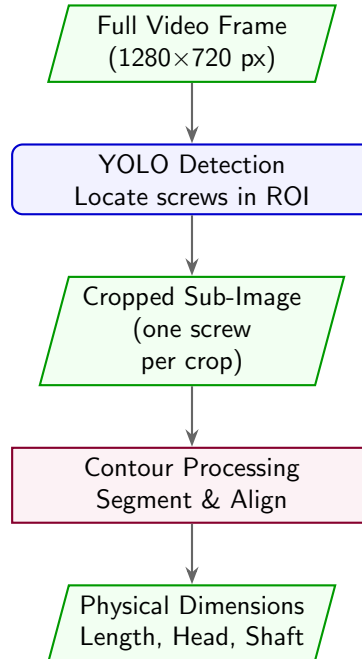


Figure 2: Step-by-step data flow from raw frame to physical screw measurements.

3.2 Step 1 — Capture and Undistort the Frame

The camera produces a raw 1280×720 BGR image. Before any processing, lens distortion is corrected using pre-calibrated camera parameters. This ensures that straight edges in the real world appear straight in the image, which is essential for accurate measurements.

3.3 Step 2 — Detect Screws with YOLO

The YOLO object detection model scans the ROI portion of the undistorted frame and draws a tight bounding box around each detected screw. An example detection is shown in Figure 3.

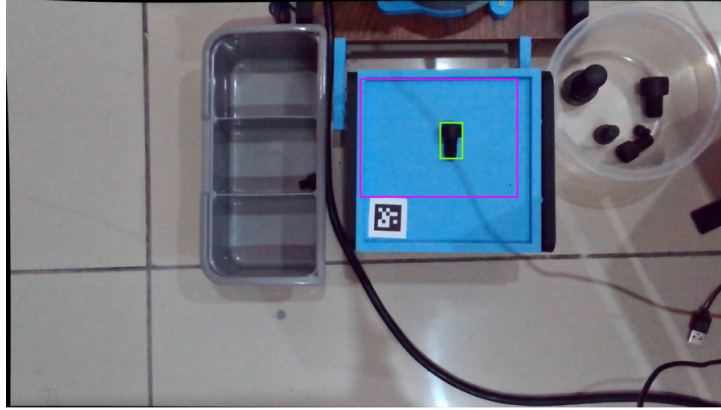


Figure 3: A screw detected inside the ROI, with its bounding box overlaid.

3.4 Step 3 — Crop the Sub-Image

Each detected bounding box is used to cut out a small patch of the image centered on that screw, with a few pixels of padding around the edges. Only this cropped sub-image is passed to the next stage, so the geometry analysis is not confused by other screws or background clutter.

3.5 Step 4 — Isolate the Screw Silhouette

The goal of this step is to produce a clean binary mask — a black-and-white image where the screw is white and everything else is black. The following sequence of image processing operations achieves this robustly, even under challenging lighting:

1. **Contrast Enhancement (CLAHE):** The grayscale image is enhanced using Contrast Limited Adaptive Histogram Equalization on an 5×5 local tile grid. This sharpens edges on metallic surfaces without amplifying bright reflective spots.
2. **Edge-Preserving Blur (Bilateral Filter):** Smooths out shadow gradients and background noise while keeping the hard outer boundary of the screw sharp.
3. **Automatic Thresholding (Otsu’s Method):** Automatically finds the best pixel value to split the image into foreground (screw) and background, producing a binary mask.
4. **Morphological Cleanup:**
 - An *opening* operation removes small stray blobs (e.g., detached shadows or dirt).
 - A *closing* operation fills small holes inside the screw mask (e.g., caused by metallic reflections).

The largest contour found in the resulting binary mask is taken as the screw’s outline (polygon).

3.6 Step 5 — Align and Measure the Screw

Because screws can lie at any angle, the mask must be rotated before measuring. The steps are:

1. **Find the orientation:** Fit the tightest possible rectangle around the screw contour to extract its angle θ and overall bounding dimensions.
2. **Rotate to horizontal:** The canvas is expanded so no part of the screw gets clipped, then an affine rotation is applied to align the screw’s long axis horizontally.

3. **Build a width profile:** The aligned binary mask is compressed into a 1D profile by counting white pixels column by column:

$$f(x) = \sum_y \frac{\text{Mask}(x, y)}{255} \quad (3)$$

This gives the width of the screw (in pixels) at every position along its length. The total length in pixels is the number of non-zero columns.

3.7 Step 6 — Extract Head and Shaft Diameters

With the 1D width profile $f(x)$ in hand, the system separates the wide screw head from the narrower shaft.

Let $W_{\max} = \max_x f(x)$ be the maximum width in the profile.

Head Diameter. The head is identified as the region where the screw is at least 75% of its maximum width:

$$\Omega_{\text{head}} = \{x \mid f(x) \geq 0.75 \cdot W_{\max}\} \quad (4)$$

To avoid errors from chamfered or uneven edges, the head diameter is taken as the *median* width within this region:

$$D_{\text{head, px}} = \begin{cases} \text{median}(\{f(x) \mid x \in \Omega_{\text{head}}\}) & \text{if } \Omega_{\text{head}} \neq \emptyset \\ W_{\max} & \text{otherwise} \end{cases} \quad (5)$$

Shaft Diameter. The shaft is the remaining portion of the profile, excluding the head region and the last 15% of the profile (the tapered tip). The boundary that discards the tip is:

$$x_{\text{cut}} = \max(1, \lfloor L \cdot (1 - 0.15) \rfloor) \quad (6)$$

The shaft candidate set is then:

$$\mathbf{w}_{\text{cand}} = \{f(x) \mid x \notin \Omega_{\text{head}} \wedge x < x_{\text{cut}}\} \quad (7)$$

When enough data points are available ($|\mathbf{w}_{\text{cand}}| \geq 3$), the candidates are sorted and the lower half is kept to focus on the root diameter rather than the thread crests:

$$\mathbf{w}_{\text{bottom}} = \left\{ \mathbf{w}_{\text{sorted}}[i] \mid 1 \leq i \leq \left\lfloor \frac{|\mathbf{w}_{\text{cand}}|}{2} \right\rfloor \right\} \quad (8)$$

$$D_{\text{shaft, px}} = \text{median}(\mathbf{w}_{\text{bottom}}) \quad (9)$$

Converting to millimeters. Both pixel diameters are converted to physical units using the ArUco scale factor κ :

$$D_{\text{head, mm}} = D_{\text{head, px}} \cdot \kappa, \quad D_{\text{shaft, mm}} = D_{\text{shaft, px}} \cdot \kappa \quad (10)$$

The processing steps and extracted geometry are visualized in Figures 4 and 5.

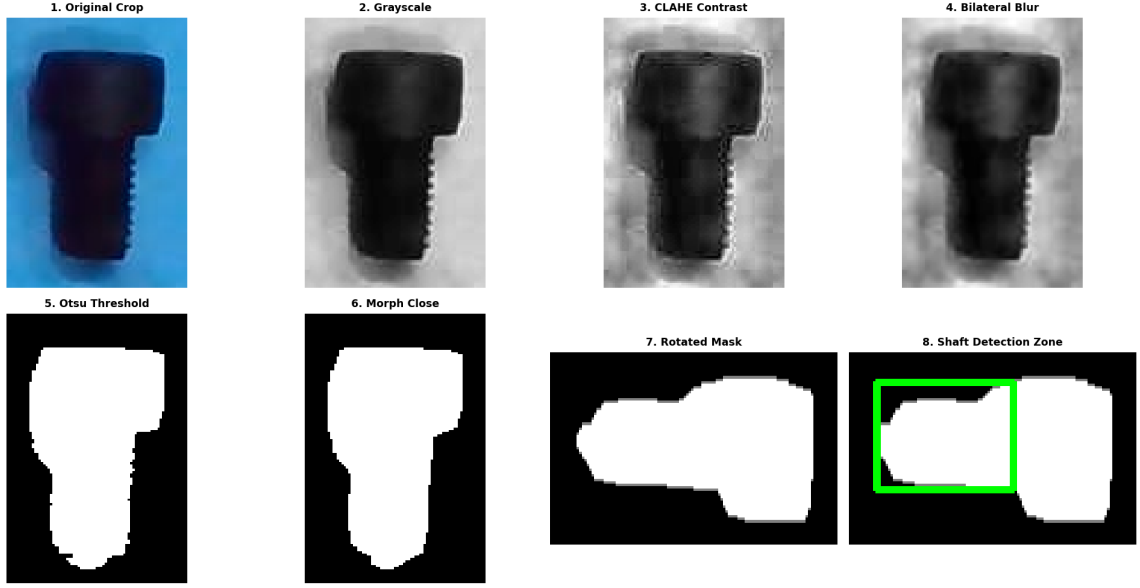


Figure 4: Intermediate outputs from each contour processing stage for screw in Figure 3.

3.8 Step 7 — Classify the Screw Size

The measured head and shaft diameters are compared against a lookup table of standard metric screws (M6 through M20). Because screw length is unreliable under perspective distortion, it is excluded from the comparison. Instead, a weighted error score E is computed for each database entry:

$$e_{\text{shaft}} = \left(\frac{D_{\text{shaft, mm}} - \mu_{\text{shaft}}}{\mu_{\text{shaft}}} \right)^2, \quad e_{\text{head}} = \left(\frac{D_{\text{head, mm}} - \mu_{\text{head}}}{\mu_{\text{head}}} \right)^2 \quad (11)$$

$$E = 0.70 \cdot e_{\text{shaft}} + 0.30 \cdot e_{\text{head}} \quad (12)$$

The shaft diameter is given a higher weight (70%) because it directly defines the metric thread size; the head diameter provides secondary confirmation.

The screw is assigned to the size with the lowest error, but only if that error is below a confidence threshold of $\tau = 0.15$. If no size matches closely enough, the screw is flagged as unknown to prevent the robot from picking it up incorrectly:

$$\text{Classification} = \begin{cases} \arg \min(E) & \text{if } \min(E) \leq 0.15 \\ \text{"M??"} & \text{if } \min(E) > 0.15 \quad (\text{unknown} - \text{skip for safety}) \end{cases} \quad (13)$$

```

--- Screw 1 Geometry ---
Length:          97 px (28.93 mm)
Head Diameter: 58.00 px (17.30 mm)
Shaft Diameter: 33.00 px (9.84 mm)
Rotation Angle: -90.00 degrees
**Standard Size: M10**

```

Figure 5: Final physical measurements extracted from the screw geometry shown in Figure 4.

4 Part 2: Screw Localization in 3D Space

Once a screw is classified, its pixel position must be converted into real-world coordinates so the robot arm knows where to move.

4.1 Coordinate Mapping Overview

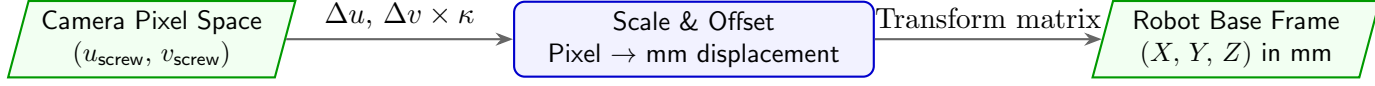


Figure 6: How pixel coordinates are converted into physical robot coordinates.

4.2 Calculation Steps

1. **Pixel displacement from the anchor:** The center of the screw's bounding box $(u_{\text{screw}}, v_{\text{screw}})$ is compared to the center of the ArUco marker $(u_{\text{anchor}}, v_{\text{anchor}})$:

$$\Delta u = u_{\text{screw}} - u_{\text{anchor}}, \quad \Delta v = v_{\text{screw}} - v_{\text{anchor}} \quad (14)$$

2. **Convert pixel offsets to millimeters:**

$$dx_{\text{cam}} = \Delta u \cdot \kappa, \quad dy_{\text{cam}} = \Delta v \cdot \kappa \quad (15)$$

3. **Align camera axes with robot axes:** The camera's horizontal and vertical axes do not necessarily match the robot's X and Y axes. A fixed rotation matrix swaps and reorders the displacements accordingly:

$$\begin{bmatrix} \Delta X \\ \Delta Y \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} dx_{\text{cam}} \\ dy_{\text{cam}} \end{bmatrix} \quad (16)$$

4. **Compute the final 3D position:** The robot-frame displacements are added to the known physical position of the ArUco marker $\mathbf{P}_{\text{marker}} = (X_m, Y_m, Z_m)$. A fixed vertical offset Z_{offset} accounts for the height difference between the marker plane and the screw surface:

$$X = X_m + \Delta X \quad (17)$$

$$Y = Y_m + \Delta Y \quad (18)$$

$$Z = Z_m + Z_{\text{offset}} \quad (19)$$

The resulting (X, Y, Z) coordinates are passed directly to the inverse kinematics solver to compute joint angles for the robot arm.

```

Base marker detected. Anchor at (u=675.0, v=384.5), Pixel Scale: 0.447 mm/px

--- Starting ROI Vision Profiling Loop ---
Measured Pixel Offsets: Delta u = +113 px, Delta v = -136 px
Calculated Mm Offsets: dx cam = +50.6 mm, dy cam = -60.7 mm
Resulting Physical World Coordinate Target Lock: X=184.3, Y=5.6, Z=108.6
  
```

Figure 7: Example output showing the computed localization coordinates for the screw in Figure 3.